# 数字图像处理

## 实验指导手册

版本：课程编号：B170031026

任课教师：周富强

北京航空航天大学仪器科学与光电工程学院

# 实验要求

1. 本手册共设置 7 个实验题目（第 1-7 章），任意选取其中 3 个实验题目完成即可。
2. 实验在平时抽时间进行编程，编程语言可以选择 C++，Matlab，Python 任何一种，也可以选择其他编程语言。实验中遇到问题，可以随时通过微信群或者课程网站进行交流。
3. 完成实验报告，要求对实验结果进行详细分析。可提交编写的应用程序。可以组团进行，最多 4 人一组，要求明确组员完成的任务，并在实验报告中说明。
4. 实验报告撰写要求：正文为小四号字体，单倍行距，页边距各为 20mm。建立个人文件夹（命名：课程 ID-姓名，将实验报告电子版（word 或 pdf 格式）、实验程序（应用或数据处理）源代码、其他材料数据放到文件夹内，压缩成单个文件，命名：课程 ID—姓名.rar, 通过电子邮件提交到：13929259@qq.com。
5. 实验报告应该包括：封面，目录，摘要，任务分工，实验目的，实验方法，实验结果及分析，实验总结及建议。请安装给定模版撰写实验报告。

《数字图像处理》教学团队

Mar 2, 2026

# 目录

**第 1 章 使用局部直方图统计的图像增强** **1**

**第 2 章 频率域滤波器设计及应用** **7**

**第 3 章 豆子自动计数** **11**

**第 4 章 焊缝区域分割** **13**

**第 5 章 基于特征的多目标提取** **16**

**第 6 章 瓶装液位检测** **21**

**第 7 章 气泡自动计数** **24**

**第 8 章 附录 1：基于 OpenCV 的轮廓和轮廓特征** **27**

8.1 寻找轮廓 . . . . . 27

8.2 绘制轮廓 . . . . . 28

8.3 轮廓层级 . . . . . 28

8.4 轮廓的特征 . . . . . 30

# 第 1 章 使用局部直方图统计的图像增强

本次实验主要依据《数字图像处理（第 4 版）》（冈萨雷斯著）中的相关理论，重点研究直方图统计方法在图像增强中的应用。实验将对比全局直方图均衡化与局部处理方法的差异，并通过基于统计量的局部增强算法，针对性地提升图像暗区域的细节可见度。

实验内容涵盖算法原理验证、参数影响分析以及针对特定图像特性的编程优化。通过查阅资料，学生需掌握不同的像素邻域遍历方法，分析其运行效率；并通过调整关键参数，观察图像增强效果的变化规律，最终实现对含隐藏细节图像的有效增强。

## 一、实验目的

1. 理解直方图的定义、物理意义及其在图像分析中的作用；
2. 掌握全局直方图均衡化的原理及其编程实现方法；
3. 理解局部直方图均衡化相对于全局方法的优缺点；
4. 掌握基于局部直方图统计量（均值和方差）的图像增强算法；
5. 掌握图像像素及其邻域遍历的高效编程方法。

## 二、实验方法

本实验选取两幅典型图像进行分析：第一幅为图 1.1 所示的“钨丝图像”，第二幅为图 1.2 所示的“嵌入隐藏物体的方形图像”。

以图 1.1 的钨丝图像为例，该图像中央区域亮度适中，结构清晰；但图像右侧（暗侧）存在另一根钨丝结构，由于亮度极低，其细节几乎不可见。针对此类问题，利用全局处理往往难以兼顾亮暗区域，因此需要采用局部增强技术。

基于直方图统计的图像增强方法，利用局部区域的灰度分布特征（均值和方差）来指导像素变换。核心思想是通过计算局部均值 $m_{S_{xy}}$ 判断区域明暗，计算局部方差 $\sigma_{S_{xy}}$ 判断区域对比度，从而设定特定的增强规则。实验将从以下三个方面展开：

1. 遍历局部区域的方法（算法效率分析）；
2. 增强算法中各参数值（$k_0, k_1, k_2, k_3, C$）的选取与影响；
3. 针对特定图像特性的算法优化（如仅处理暗区域）。

# 三、实验内容

## (一) 全局直方图均衡化与局部直方图均衡化

直方图提供了图像灰度分布的统计信息。令离散随机变量 $r_k$ 表示区间 $[0, L-1]$ 内的灰度值，$p(r_k)$ 为对应的归一化直方图分量，即图像中出现灰度 $r_k$ 的概率估计。

图 1.2 所示图像大小为 $512 \times 512$ 像素，8 比特深度。该图像主体由背景中的 5 个黑色方格组成，且存在轻微噪声。关键在于，这些黑色方格中嵌入了某些物体，由于背景与物体对比度极低，这些物体在原始图像中几乎不可见。针对该图像，完成以下编程及实验任务：

1. 编程实现**全局直方图均衡化**，对图 1.2 所示图像进行处理，分析处理后的图像效果及直方图变化。
2. 选取邻域大小为 $3 \times 3$，编程实现**局部直方图均衡化**（滑动窗口法），对图 1.2 所示图像进行处理，对比全局方法的结果，分析局部增强在细节显现上的优势与可能存在的噪声放大问题。

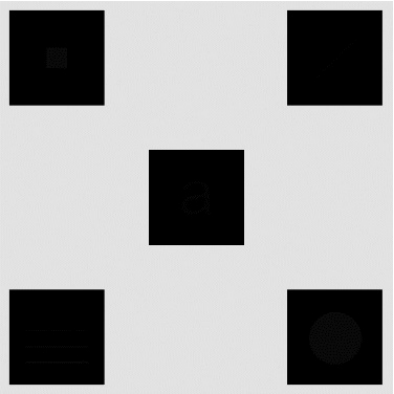


**图 1.1:** 图 1.2 嵌入隐藏物体的方形图像

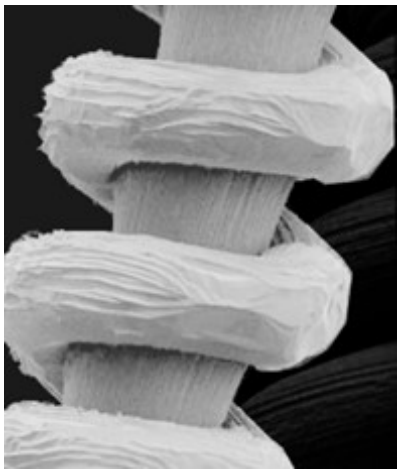


**图 1.2:** 图 1.1 钨丝 SEM 图像

## (二) 使用直方图统计量增强图像

对于灰度级在区间 $[0, L-1]$ 内的图像，灰度值 $r$ 相对于其均值 $m$ 的第 $n$ 阶矩定义为：

$$\mu_n = \sum_{i=0}^{L-1} (r_i - m)^n p(r_i) \tag{1.1}$$

其中，均值 $m$ 为：

$$m = \sum_{i=0}^{L-1} r_i p(r_i) \tag{1.2}$$

均值是平均灰度的测度，而方差（二阶矩）$\sigma^2 = \mu_2$ 则是图像对比度的测度。

**1. 局部均值与方差**

令 $(x, y)$ 是图像中任意像素的坐标，$S_{xy}$ 是以 $(x, y)$ 为中心的邻域（例如 $m \times n$ 的矩形窗口）。该邻域的直方图为 $p_{S_{xy}}(r_i)$。邻域的局部均值定义为：

$$m_{S_{xy}} = \sum_{i=0}^{L-1} r_i p_{S_{xy}}(r_i) \tag{1.3}$$

邻域的局部方差定义为：

$$\sigma^2_{S_{xy}} = \sum_{i=0}^{L-1} (r_i - m_{S_{xy}})^2 p_{S_{xy}}(r_i) \tag{1.4}$$

局部均值反映了邻域内的平均亮度，局部方差反映了邻域内的对比度。

**2. 增强规则设计**

基于局部统计量的增强可以根据图像的局部特征动态调整处理策略。

- **亮度判断**：将局部均值 $m_{S_{xy}}$ 与全局均值 $M_G$ 比较。若 $k_0 M_G \leq m_{S_{xy}} \leq k_1 M_G$（$k_0, k_1$ 为非负常数且 $k_0 < k_1$），则认为该区域亮度符合特定要求（如较暗区域），将其作为候选处理区域。
- **对比度判断**：将局部标准差 $\sigma_{S_{xy}}$ 与全局标准差 $\sigma_G$ 比较。若 $k_2 \sigma_G \leq \sigma_{S_{xy}} \leq k_3 \sigma_G$（$k_2, k_3$ 为非负常数且 $k_2 < k_3$），则认为该区域对比度符合特定要求（如低对比度区域），对其进行增强。

**3. 增强变换公式**

设 $f(x, y)$ 为输入图像，$g(x, y)$ 为增强后图像。对于满足上述条件的像素点，按如下公式进行变换：

$$g(x, y) = \begin{cases} C \cdot f(x, y), & \text{若} k_0 M_G \leq m_{S_{xy}} \leq k_1 M_G \text{ 且} k_2 \sigma_G \leq \sigma_{S_{xy}} \leq k_3 \sigma_G \\ f(x, y), & \text{其他} \end{cases} \tag{1.5}$$

其中，$C$ 为规定的常数增益。当 $C > 1$ 时，可提升符合条件的暗区域像素值，从而显示出隐藏细节。

**4. 实验任务**

根据上述方案，完成以下实验内容：

1. 编写程序实现：图像全局直方图统计（计算 $M_G, \sigma_G$）、局部滑动窗口直方图统计（计算 $m_{S_{xy}}, \sigma_{S_{xy}}$）以及基于上述公式的选择性局部增强算法。

2. **参数分析**：针对图 1.1 所示钨丝图像，通过实验探讨关键参数的选择方法。重点分析参数 $k_0, k_1$（确定暗区范围）、$k_2, k_3$（确定低对比度范围）、增益常数 $C$ 以及局部区域 $S_{xy}$ 大小对最终增强图像质量的影响。例如，可尝试设置 $k_0 = 0, k_1 = 0.5$ 以筛选暗区，设置 $k_2 = 0, k_3 = 0.2$ 以筛选平坦区域。
3. **特定目标增强**：利用编写的程序对图 1.1 钨丝图像的右侧暗区进行增强。目标是使右侧不可见的钨丝结构变得清晰可辨，同时注意抑制背景噪声的过度放大。

## (三) 参考代码

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

# 计算局部均值和标准差
def local_mean_stddv(input_img, x, y):
    img=[[] for i in range(3)]
    for i in range(3):
        for j in range(3):
            img[i].append(input_img[x-1+i][y-1+j])
    img_np = np.array(img)
    (mean , stddv) = cv.meanStdDev(img_np)
    return mean, stddv

# 使用直方图统计量增强局部图像
def HistogramStatisticsEnhance(input_img):
    # 初始化
    output_image = input_img
    # 定义参数
    k0 = 0
    k1 = 0.25
    k2 = 0
    k3 = 0.1
    C = 20
    # 创建空列表
```

```
    means = [[] for i in range(512)]
    stddvs = [[] for i in range(512)]
    # 计算全局均值和标准差
    (m_G, sigma_G) = cv.meanStdDev(input_img)
    # 计算每个元素的 3 邻域的均值与标准差
    for i in range(0, 512):
        for j in range(0, 512):
            (mean, stddv) = local_mean_stddv(input_img, i+1, j+1)
            means[i].append(mean)
            stddvs[i].append(stddv)
    # 代入分段函数
    for i in range(0, 512):
        for j in range(0, 512):
            if (k0*m_G <= means[i][j] and means[i][j] <= k1*m_G) and (k2
                *sigma_G <= stddvs[i][j] and stddvs[i][j] <= k3*sigma_G):
                output_image[i, j] = C*input_img[i][j]
    return output_image

def main():
    # 读取图片
    src = cv.imread("Fig0326(a)(embedded_square_noisy_512).tif")
    # 转化为灰度图
    gray = cv.cvtColor(src, cv.COLOR_BGR2GRAY)
    # 显示灰度图
    cv.imshow("gray␣image", gray)
    # 绘制直方图
    plt.hist(gray.ravel(),256,[0,256]); plt.show()
    # 扩充边界，cv.BORDER_REPLICATE: 使用最边界的像素值代替
    gray1 = cv.copyMakeBorder(gray,1,1,1,1,cv.BORDER_REPLICATE)
    # 使用直方图统计量增强局部图像
    # gray1.dtype = "float16"
    enhanced_img = HistogramStatisticsEnhance(gray1)
    # 显示使用直方图统计量增强局部的图像
    cv.imshow("enhanced_img", enhanced_img)
    cv.waitKey(0)
```

```
    cv.destroyAllWindows()
    cv.imwrite("enhanced_img.tif", enhanced_img)

if __name__ == "__main__":
    main()
```

# 第 2 章 频率域滤波器设计及应用

本次实验主要依据《数字图像处理（第 4 版）》（冈萨雷斯著）中的相关理论，重点研究频率域滤波技术在图像修复中的应用。实验旨在通过设计陷波滤波器，消除图像中的周期性噪声干扰。

实验涵盖两类典型的周期性噪声问题：遥感图像中的水平扫描线干扰，以及印刷/扫描图像中的莫尔条纹和正弦干扰。通过分析图像的傅里叶频谱，识别噪声在频率域中的分布特征，设计针对性的滤波器以阻断特定频率分量，从而实现图像去噪。实验要求采用模块化编程思想，通过交互式方式设计滤波器，直观探讨滤波器参数对图像复原效果的影响。

## 一、实验目的

1. 理解并掌握陷波滤波器的基本定义、原理及频率特性；
2. 掌握周期性噪声干扰在空间域与频率域的特征及其相互关系；
3. 学习利用编程工具（如 MATLAB/Python）设计并实现理想陷波滤波器与巴特沃斯陷波滤波器；
4. 直观体会陷波滤波器消除周期性干扰的效果，并与低通滤波等方法进行对比分析。

## 二、实验方法

根据实验要求，需设计特定类型的频率域滤波器。核心在于分析含噪图像的频谱，确定噪声干扰点的位置（如周期性干扰通常表现为频谱中对称的一对或多个高亮尖峰），从而设计滤波器函数 $H(u,v)$ 对这些频点进行抑制。

建议采用交互式界面（如 GUI）辅助设计。通过鼠标在频谱图上选取干扰点位置，程序自动计算滤波器参数并实时显示处理结果。程序的主要流程设计如下：

1. **图像读取与填充**：读入原始图像 $f(x,y)$，为避免频率域折叠效应，需对图像进行填充（Padding），将其扩展至 $P \times Q$ 大小（通常 $P \geq 2M, Q \geq 2N$）。
2. **中心化变换**：构造中心化矩阵 $(-1)^{x+y}$ 并与图像相乘，实现傅里叶变换的频谱中心化。
3. **DFT 计算**：计算图像的离散傅里叶变换（DFT），得到复数频谱 $F(u,v)$。

4. **滤波器设计**：根据用户交互选取的噪声点坐标 $(u_k, v_k)$，生成对应的陷波滤波器 $H(u,v)$。滤波器应在噪声点处为 0（或极小值），在其他位置为 1。
5. **频域滤波**：将滤波器与图像频谱相乘：$G(u,v) = H(u,v) \cdot F(u,v)$。
6. **反变换与后处理**：对 $G(u,v)$ 进行傅里叶反变换（IDFT），取实部，并进行中心反平移，截取原始图像大小，得到最终的处理图像 $g(x,y)$。

## 三、实验内容

### (一) 使用低通与陷波滤波消除水平扫描线

图 2.1 是一幅显示了墨西哥湾（暗色）和佛罗里达（亮色）的甚高分辨率辐射计（VHRR）图像，大小为 $808 \times 754$ 像素。由于传感器沿扫描方向的响应特性，图像中存在突出的水平扫描线噪声。这类噪声在频谱中通常表现为垂直中心轴上的高频分量。

1. **低通滤波实验**：选取合适的截止频率，设计高斯低通滤波器对图 2.1 进行处理。观察处理结果，分析低通滤波在消除扫描线的同时对图像边缘细节产生的模糊效应。
2. **陷波滤波实验**：分析频谱图，设计一种陷波滤波器（或多个陷波滤波器的组合），精确阻断频谱中对应扫描线干扰的频率分量。处理图 2.1，重点观察陷波滤波器在消除扫描线的同时，如何更好地保留图像的边缘细节，并与低通滤波结果进行对比分析。

### (二) 使用陷波滤波减少莫尔波纹模式

图 2.2 是一幅扫描报纸的图像，由于印刷网点与扫描采样频率之间的干涉，图像中出现了明显的莫尔波纹模式。这种周期性干扰在频谱中通常表现为特定的周期性亮点对。

请设计一个**巴特沃斯陷波带阻滤波器**，通过交互方式定位频谱中的干扰点，调整滤波器阶数和半径参数，消除图像中的莫尔条纹，同时尽量保留文字和图片细节。

### (三) 使用陷波滤波增强恶化的卡西尼土星图像

图 2.3 展示了环绕土星的土星环图像，图像大小为 $674 \times 674$ 像素。由于数字化过程中视频信号叠加了交流（AC）干扰，图像中存在垂直的正弦状干扰模

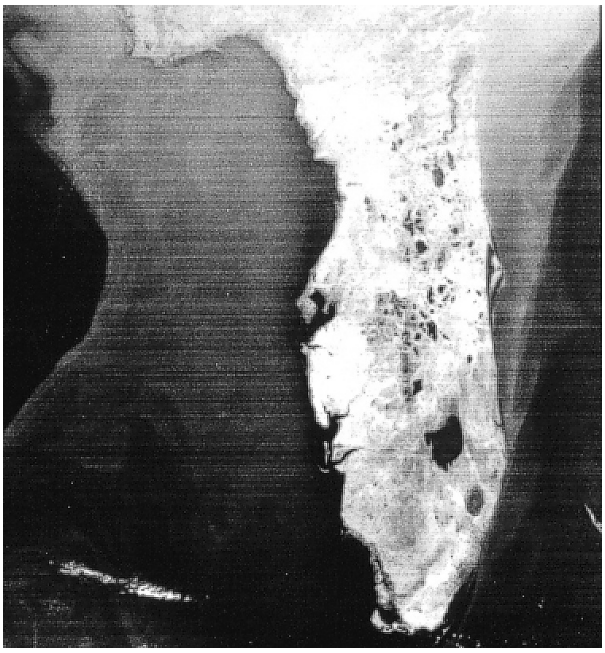


**图 2.1:** 图 2.1 带有突出水平扫描线的卫星图像（VHRR）

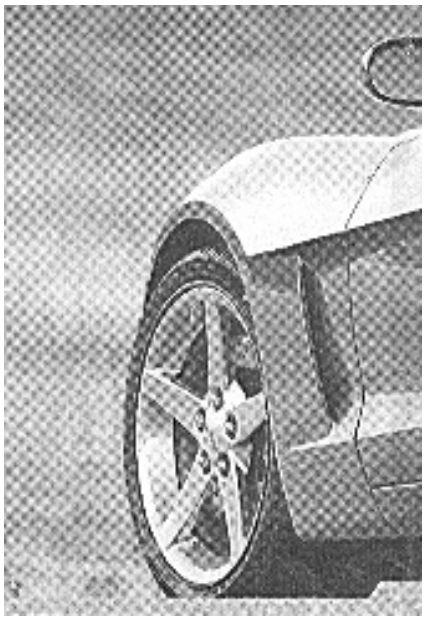


**图 2.2:** 图 2.2 带有莫尔波纹的扫描报纸图像

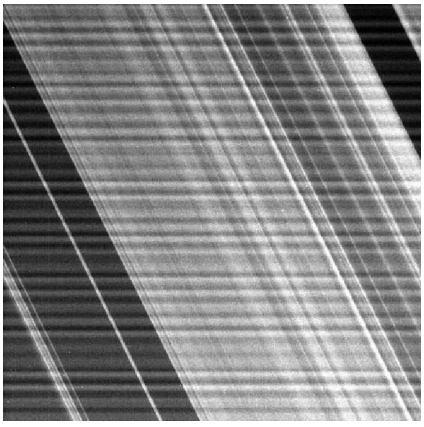


**图 2.3:** 图 2.3 具有近似周期性干扰的卡西尼土星图像

式，严重影响了土星环的观测。

请根据频谱特征设计一种陷波带阻滤波器，分离并滤除此周期性干扰信号，恢复土星环的清晰细节。注意分析干扰信号在频谱中的对称性分布特征。

# 第 3 章 豆子自动计数

本次实验是一个综合性的图像处理项目，旨在通过解决实际的“豆子计数”问题，串联起图像处理中的关键环节。实验将综合运用图像增强、图像平滑、图像分割、数学形态学处理及特征提取等知识，训练学生设计完整图像处理算法链条的能力，并基于 OpenCV 平台实现目标检测与自动计数。

## 一、实验目的

1. 理解图像增强（如对比度调整）在改善分割效果中的前提作用；
2. 理解形态学处理（如腐蚀、膨胀、开闭运算）在去噪、分离粘连目标及填补空洞中的关键作用；
3. 理解边缘提取算法（如 Canny 算子）的原理及其在目标轮廓检测中的应用；
4. 掌握阈值分割算法（如 Otsu 大津法）的原理及其在目标与背景分离中的应用；
5. 掌握基于 OpenCV 进行综合图像处理系统设计与实现的方法。

## 二、实验方法

本实验遵循“问题驱动”的原则，采用自底向上的图像处理流程设计算法。针对豆子图像中存在的光照不均、背景噪声、豆子粘连以及豆子边缘模糊等问题，设计标准化的处理流水线。主要的算法设计步骤如下：

1. **图像预处理与增强**：读入原始图像，将其转换为灰度图像。针对光照不均的情况，可采用直方图均衡化或 Gamma 校正进行增强，提高前景（豆子）与背景的对比度。
2. **图像平滑**：分析图像噪声情况，选择合适的滤波器（如高斯滤波或中值滤波）去除噪声，同时尽量保持边缘信息。
3. **图像分割**：
   - **方法一（阈值法）**：利用 Otsu 算法（最大类间方差法）寻找最佳阈值，将图像二值化，将豆子从背景中分离出来；
   - **方法二（边缘法）**：利用 Canny 边缘检测算子提取豆子的轮廓。

4. **形态学处理**：针对二值化图像中可能存在的豆子内部空洞或背景中的孤立噪点，利用闭运算（先膨胀后腐蚀）填充目标内部空洞；针对粘连的豆子，利用腐蚀运算或分水岭算法尝试进行分离。
5. **特征提取与计数**：
   - 对于二值化结果，利用连通域分析（如 OpenCV 中的 `connectedComponents` 或 `findContours`），统计连通区域的个数。
   - 对于边缘检测结果，通过寻找闭合轮廓并统计轮廓数量。
6. **结果标注与显示**：在原始图像上通过绘制圆圈或方框标记出识别到的每一个豆子，并标注计数的总数。

## 三、实验内容

图 3.1 展示了 4 幅包含不同数量及形态的豆子图像。这些图像在光照强度、豆子排列紧密程度（是否存在粘连）以及背景复杂度上各有不同。请基于 OpenCV 设计并编写程序，完成以下任务：

1. **基础计数实现**：设计一套通用的图像处理方案，能够准确识别并统计出图 3.1 中各幅图像的豆子数量。要求程序能够适应不同数量的豆子目标。
2. **粘连问题处理**：重点针对豆子排列紧密、可能发生粘连的情况，对比直接阈值分割与引入形态学腐蚀操作（或距离变换 + 分水岭算法）后的分割效果，分析哪种方法能更准确地分离粘连豆子，从而提高计数精度。
3. **边缘与区域分割对比**：尝试分别使用“全局阈值分割”和“Canny 边缘检测 + 轮廓查找”两种技术路线进行计数，比较两种方法在对光照变化敏感度及抗噪性能方面的差异。
4. **可视化输出**：编写程序界面，使得处理结果能够直观显示，包括二值化图像、边缘检测结果以及最终在原图上标记计数的结果。

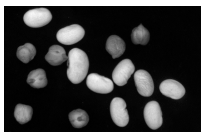


图 3.1: 图像 1

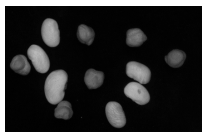


图 3.2: 图像 2

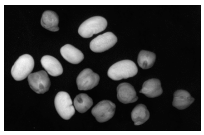


图 3.3: 图像 3

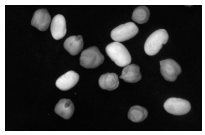


图 3.4: 图像 4

图 3.5: 待计数的豆子图像样本

# 第 4 章 焊缝区域分割

本次实验综合运用《数字图像处理》教材中基于区域生长算法的图像分割技术，针对工业 X 射线图像进行焊缝缺陷检测。实验旨在通过编程实现（可选用 C++、MATLAB 或 Python），将图像中代表裂缝和孔隙的缺陷区域从背景中精确分割出来。设计方案既可参照教科书的经典分析流程，也鼓励学生结合直方图特性自行设计优化的分割算法。

## 一、实验目的

1. 理解并掌握基于区域生长的图像分割方法的基本原理；
2. 掌握图像分割中阈值（包括双阈值）的选取与优化方法；
3. 理解区域分割中种子点的选取策略及其对分割结果的影响；
4. 对比分析区域生长分割与全局阈值分割在处理复杂背景图像时的优缺点。

## 二、实验方法

本实验的核心思想是利用图像的灰度统计特性，通过计算原始图像与种子点（或种子均值）的差异，将缺陷区域从背景中分离出来。根据分析思路的不同，可采用以下两种方法：

**1. 基于种子点的区域生长法**

- **种子点初始化**：通过分析原始图像的直方图或人工交互，在疑似缺陷区域选取一个或多个初始种子点，设定种子灰度值 $S$。
- **差值图像生成**：计算原始图像 $f(x,y)$ 与种子值 $S$ 的绝对差值图像 $D(x,y) = |f(x,y) - S|$。在缺陷区域，像素灰度与种子值接近，差值较小；在焊缝金属背景或母材区域，灰度差异较大。
- **区域生长准则**：设定一个灰度阈值 $T$。遍历图像，若某像素的差值 $D(x,y) \leq T$ 且与已生长区域相邻（4 连通或 8 连通），则将其合并到区域中。
- **迭代生长**：重复上述过程，直到区域不再扩大，最终得到分割结果。

**2. 基于双阈值的差值图像分割法（自动化方案）**

- **生成差值图像**：同上，生成绝对差值图像 $D(x,y)$。
- **直方图分析**：观察差值图像 $D(x,y)$ 的直方图。由于缺陷区域集中了大量低差值像素，而背景区域对应高差值像素，直方图通常呈现明显的双峰特

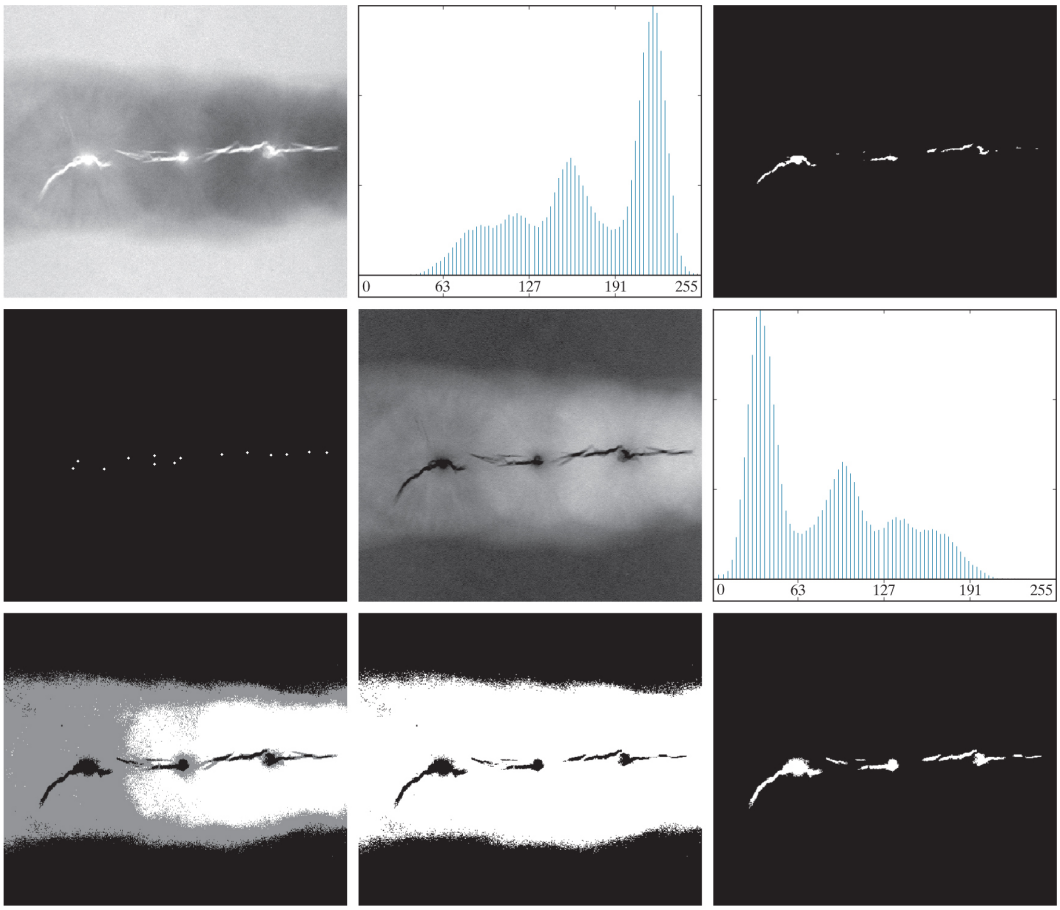
性。

- **双阈值分割**：利用 Otsu 算法（最大类间方差法）或人工选取双阈值 $T_1$ 和 $T_2$（$T_1 < T_2$），对差值图像进行阈值处理，提取出低差值区域，即缺陷区域。

## 三、实验内容

图 4.1 展示了一条带有缺陷的焊缝 X 射线图像及其处理过程示意图。图像背景为较亮的母材，中间有一条较暗的焊缝，而焊缝中存在的裂缝和孔隙表现为局部的灰度异常。请根据以下步骤设计并实现算法：

1. **图像读取与预处理**：读入图 4.1(a) 所示的 8 比特 X 射线图像。分析其直方图 [图 4.1(b)]，观察焊缝区域、缺陷区域及母材背景的灰度分布特征。
2. **种子点选取策略**：
   - 实现程序交互功能，允许用户在图像中点击选取初始种子点 [参考图 4.1(c)]，或通过灰度范围自动筛选种子点。
   - 对选取的种子点进行统计，计算种子区域的平均灰度值，作为后续计算的基准。
3. **差值图像与分割算法实现**：
   - 计算原始图像与种子值的绝对差值图像 [参考图 4.1(e)]。
   - 分析差值图像的直方图 [参考图 4.1(f)]，选取合适的阈值。尝试使用单阈值和双阈值分别处理差值图像 [参考图 4.1(g) 和]。
   - 或者，实现基于邻域一致性的区域生长算法，从种子点开始逐步“吞噬”相邻的相似像素。
4. **结果分析与优化**：
   - 输出最终的分割结果图像 [参考图 4.1(i)]，其中缺陷区域应被清晰标记出来（如置为白色或特定颜色）。
   - 对比不同阈值参数下，分割出的缺陷区域是否完整、是否包含误判的背景噪声。
   - 分析区域生长算法如何避免将背景中灰度相似的噪声误判为缺陷。




a b c
d e f
g h i

**图 4.1:** 焊缝缺陷区域分割全过程：(a) 缺陷焊缝的 X 射线图像；(b) 直方图；(c) 初始种子图像；(d) 最终种子图像（点已被放大）；(e) 种子值（255）与图 (a) 的差的绝对值；(f) 图 (e) 的直方图；（g）使用双阈值对图像进行阈值处理后得到的差值图像；(h) 使用最小的双阈值对图像进行阈值处理后得到的差值图像；(i) 采用区域生长得到的分割结果

# 第 5 章 基于特征的多目标提取

本次实验是一个综合性的图像分析项目，旨在解决图像中多目标的分割与分类问题。实验将综合运用边缘提取、阈值分割、数学形态学处理等图像预处理方法，实现目标的有效分割；在此基础上，利用提取的边界或区域数据计算目标的几何特征参数（如圆度、长宽比、面积等），进而实现基于特征的目标分类与识别。

## 一、实验目的

1. 进一步熟悉并掌握图像分割的基本方法（阈值分割、边缘检测等）；
2. 理解并掌握基于区域和边界特征提取的基本方法；
3. 能够综合运用多种图像处理技术，编程实现复杂场景下的多目标提取与分类；
4. 掌握利用 OpenCV 或 MATLAB 进行图像特征度量与形状分析的编程技巧。

## 二、实验方法

本实验采用“预处理—分割—特征计算—分类决策”的标准流程设计算法：

1. **图像预处理**：读入原始图像，进行灰度化处理。根据图像质量选择合适的滤波算法（如中值滤波）去除噪声，并利用直方图均衡化或对比度拉伸增强前景与背景的对比度。
2. **图像分割**：采用全局阈值（如 Otsu 法）或自适应阈值法将图像转换为二值图像。利用形态学操作（如开运算去除小噪点、闭运算填充目标内部孔洞）优化分割结果。
3. **特征提取**：对分割后的每一个连通域进行标记。利用轮廓跟踪或区域属性计算，提取以下特征参数：
   - **几何特征**：面积（Area）、周长（Perimeter）、质心（Centroid）、外接矩形（Bounding Box）。
   - **形状特征**：圆度（Circularity，$4\pi A/P^2$）、长宽比（Aspect Ratio）、紧凑度等。
4. **目标分类**：根据提取的特征参数设定分类规则。例如，设定圆度阈值 $T$，

若目标的圆度大于 $T$ 则判定为圆形物体，否则判定为其他形状物体。在原图上标记出分类结果（如用不同颜色的框或文字标注）。

## 三、实验内容

如图 5.1 所示，图像中包含多种不同形状的目标（如圆形的药丸和矩形的胶囊）。请使用 OpenCV（C++/Python）或 MATLAB 完成以下任务：

1. 设计并实现图像处理算法，将图中的各个目标从背景中完整分割出来。
2. 针对每一个目标，计算其面积和周长，进而计算其“圆度”特征参数：
$$\text{Circularity} = \frac{4\pi \times \text{Area}}{\text{Perimeter}^2}$$
（注：对于理想圆，该值为 1；对于其他形状，该值小于 1）。
3. 根据圆度特征设定阈值，将图像中的目标分类为“圆形”和“非圆形”两类。
4. 在原始图像上对分类结果进行可视化显示：例如，用绿色圆圈圈出圆形目标，用红色矩形框出非圆形目标，并标注计算得到的圆度数值。

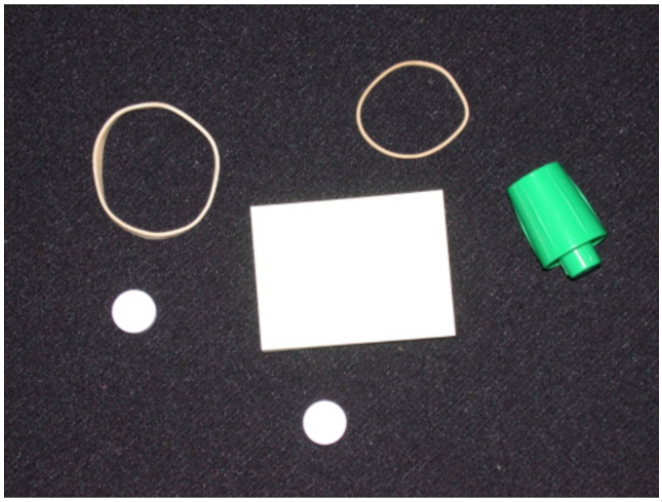


图 5.1: 图 5.1 待分类的多目标图像

# 附录：基于 MATLAB 编程的多目标分割与分类示例

本附录展示如何使用 MATLAB 的图像处理工具箱，通过边界跟踪例程 `bwboundaries` 根据对象的圆度对其进行分类。

## 步骤 1：读取图像

读入包含药丸和胶囊的示例图像。

```
RGB = imread('pillsetc.png');
imshow(RGB);
title('原始图像');
```

## 步骤 2：图像阈值化

将彩色图像转换为灰度图像，并进行二值化处理，为边界跟踪做准备。

```
I = rgb2gray(RGB);
bw = imbinarize(I);
imshow(bw);
title('二值化图像');
```

## 步骤 3：形态学去噪与填充

使用形态学函数删除不属于关注对象的微小噪点，并填充目标内部的间隙。

```
% 删除包含少于30个像素的所有对象（去噪）
bw = bwareaopen(bw, 30);

% 使用形态学闭运算填充笔帽或目标中的细小间隙
se = strel('disk', 2);
bw = imclose(bw, se);

% 填充所有孔洞，确保regionprops能准确估计封闭区域的面积
bw = imfill(bw, 'holes');
imshow(bw);
title('形态学处理后的图像');
```

## 步骤 4：查找边界

使用 `bwboundaries` 函数获取所有外边界。选项`'noholes'` 用于加速处理，忽略内部轮廓（如孔洞边缘）。

```
[B, L] = bwboundaries(bw, 'noholes');
```

显示标签矩阵并绘制每个边界，以验证分割效果。

```
imshow(label2rgb(L, @jet, [.5 .5 .5]));
hold on;
for k = 1:length(B)
boundary = B{k};
plot(boundary(:,2), boundary(:,1), 'w', 'LineWidth', 1.5);
end
title('边界标记可视化');
```

## 步骤 5：计算圆度并分类

估计每个对象的面积和周长，构建圆度度量指标，并根据阈值判定是否为圆形目标。

```
% 使用regionprops获取所有对象的面积和质心
stats = regionprops(L, 'Area', 'Centroid');

% 设定圆度阈值，只有接近圆的对象才会大于此值
threshold = 0.94;

% 遍历所有边界
for k = 1:length(B)
% 获取第k个边界的坐标
boundary = B{k};

% 计算周长：计算边界点之间的欧氏距离之和
delta_sq = diff(boundary).^2;
perimeter = sum(sqrt(sum(delta_sq, 2)));

% 获取面积
area = stats(k).Area;

% 计算圆度指标：4*pi*Area / Perimeter^2
```

```
metric = 4 * pi * area / perimeter^2;

% 格式化显示数值
metric_string = sprintf('%2.2f', metric);

% 标记高于阈值的对象（认为是圆形）
if metric > threshold
centroid = stats(k).Centroid;
plot(centroid(1), centroid(2), 'ko');
end

% 在物体旁边显示圆度数值
text(boundary(1,2) - 35, boundary(1,1) + 13, ...
metric_string, 'Color', 'y', ...
'FontSize', 14, 'FontWeight', 'bold');
end

title(['Metrics␣closer␣to␣1␣indicate␣that␣the␣object␣is␣approximately
    ␣round']);
```

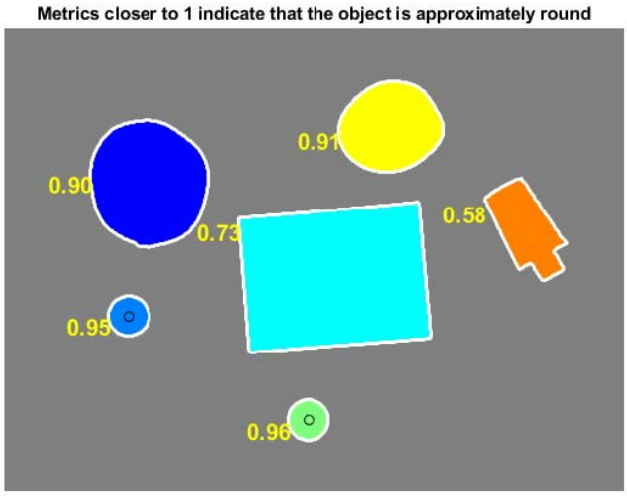



图 5.2: 图 5.2 基于圆度特征的多目标分类结果

# 第 6 章 瓶装液位检测

本次实验是一个面向工业视觉检测的综合应用项目。实验要求综合运用图像增强、形态学处理、图像分割及特征提取等技术，解决传送带上瓶装液体液位的自动检测问题。实验旨在训练学生将理论知识转化为解决实际工程问题的能力，设计能够自动判断瓶子是否装满的视觉算法。

## 一、实验目的

1. 进一步熟悉并掌握复杂背景下的图像分割方法（如边缘检测、霍夫变换）；
2. 理解基于几何特征的提取与测量方法；
3. 能够利用多种图像处理方法设计完整的视觉检测系统，解决未装满瓶子的识别问题。

## 二、实验方法

本实验采用“问题驱动”的策略，核心在于精确定位瓶子的关键几何特征（液面线和瓶颈/肩部线），并通过计算两者之间的垂直距离来实现检测。建议的算法流程设计如下：

1. **图像预处理：**读入原始图像，进行灰度化处理。利用直方图均衡化或伽马变换增强液体与背景、玻璃边缘的对比度。根据需要使用高斯平滑滤波以减少噪声。
2. **感兴趣区域（ROI）提取：**为了减少计算量并避免背景干扰，可以通过手动设定或自动检测的方法，截取包含瓶身和瓶颈的关键区域作为 ROI。
3. **边缘检测与分割：**利用 Canny 边缘检测算子提取图像中的边缘特征。重点提取液体的水平上边缘（液位线）以及瓶肩部的轮廓线。
4. **特征提取与几何计算：**
   - **液位定位：**在 ROI 的中下部搜索水平边缘，拟合直线得到液位的高度坐标 $y_{liquid}$。
   - **参考点定位：**根据瓶子轮廓特征，识别瓶颈底部与瓶身（或称肩部）的交界处，确定参考线的位置 $y_{shoulder}$。
   - **液位判定：**计算液位线与参考线之间的垂直距离 $H = |y_{liquid} - y_{shoulder}|$，或者直接比较两者的高度关系。

5. **决策与分类**：设定判定阈值。根据题意，若液位线低于瓶肩部中间点，则判定为“未装满”；否则判定为“合格”。在原图上用不同颜色（如红色标记不合格，绿色标记合格）或文字显示检测结果。

## 三、实验内容

某化工企业使用瓶子盛装工业化学品，需要设计一种视觉系统来检测瓶子是否未装满。瓶子在传送带上移动，系统配备前端照明闪光灯，可获取如图 6.1 所示的清晰静态图像。检测标准为：当液位低于瓶颈底部与瓶子肩部（侧面与倾斜面交界处）的中间点时，判定该瓶子未装满。

请根据上述描述，完成以下实验任务：

1. **问题建模与假设陈述**：在进行算法设计前，请清晰陈述您对问题的假设，这些假设可能影响您的解决方案。例如：
   - 假设相机光轴与传送带运动方向垂直，图像无透视畸变（或已校正）；
   - 假设液面在图像中呈现为近似水平的直线；
   - 假设瓶子在图像中的位置相对固定，或背景简单易于分割；
   - 假设液体与玻璃瓶壁/背景有足够的灰度对比度。
2. **算法设计与实现**：基于 OpenCV（C++/Python）或 MATLAB 设计程序，实现以下功能：
   (a). 自动检测并提取瓶子的边缘轮廓；
   (b). 准确识别液体的上表面边缘（液位线）；
   (c). 准确定位瓶颈底部与瓶肩部的交界区域，并计算其中点坐标；
   (d). 计算液位线与该中点在垂直方向上的距离，并根据距离值判断液位是否达标。
3. **结果验证**：对图 6.1 所示图像进行处理，在图像上绘制出检测到的液位线、参考中点线以及测量距离。输出“合格”或“未装满”的判断结果，并分析算法的鲁棒性。

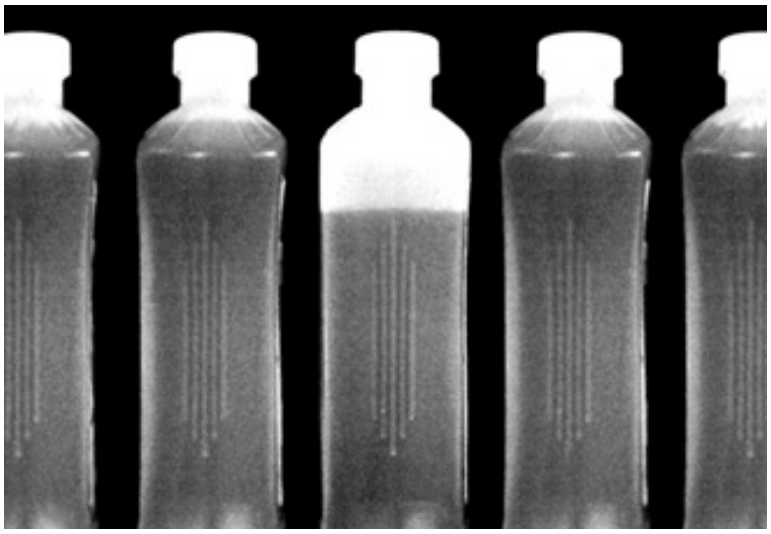


图 6.1: 瓶装液体检测图像

# 第 7 章 气泡自动计数

本次实验是一个面向工业质量控制的综合应用项目。实验要求综合运用图像增强、形态学处理、图像分割及特征提取等技术，解决液体中微小气泡的自动计数与面积比率计算问题。实验旨在训练学生处理低对比度、密集目标图像的能力，并建立图像像素与物理世界尺寸之间的映射关系。

## 一、实验目的

1. 进一步熟悉并掌握复杂背景下的图像分割方法（特别是基于形态学的分割）；
2. 理解特征提取中的形状度量（面积、周长、等效直径）及其物理意义；
3. 掌握分水岭算法或距离变换在解决目标粘连问题中的应用；
4. 能够利用多种图像处理方法设计完整的视觉检测方案，实现气泡的定量分析。

## 二、实验方法

由于气泡图像通常具有气泡边缘较暗、内部较亮（或反之）且光照不均的特点，同时可能存在大量微小气泡和气泡粘连现象，建议采用以下技术路线设计算法：

1. **图像预处理与背景校正**：读入 $800 \times 800$ 像素的 8 比特图像。利用形态学顶帽变换或基于 rolling ball 的背景减法算法，去除光照不均匀的背景干扰，突出气泡目标。
2. **图像分割**：采用全局阈值（如 Otsu 法）或自适应阈值法对背景校正后的图像进行二值化。针对分割结果中可能存在的噪声，利用 `bwareaopen`（基于面积的开运算）去除过小的噪点。
3. **粘连分离（关键步骤）**：针对图像中可能存在的粘连气泡，使用距离变换结合分水岭算法，将粘连的气泡分割为独立的个体，以提高计数精度。
4. **特征测量与统计**：
   - **面积比率**：统计所有气泡区域的像素总数 $N_{bubble}$，计算其占图像总像素 $N_{total}$ 的比例 $R = N_{bubble}/N_{total}$。

- **气泡计数**：通过连通域标记或分水岭后的区域标记，统计气泡的数量。
- **最小尺寸检测**：计算每个气泡的等效直径或像素面积，确定系统可检测到的最小气泡的像素尺寸，并结合已知物理面积换算为物理尺寸。

## 三、实验内容

某流体公司需要对其产品进行质量控制，需对液体中的气泡进行自动分析。已知成像系统获取的图像分辨率为 $800 \times 800$ 像素，深度为 8 比特，每幅图像对应的实际物理面积为 $8\text{cm}^2$。图 7.1 为典型的气泡图像。

请根据上述资料，完成以下实验任务：

1. **参数标定与最小可检测尺寸估算**：根据给定的图像分辨率与物理面积，计算像素分辨率（即 $\mu$m/pixel 或 cm/pixel）。结合图像处理算法中设定的最小去噪阈值，计算并指出本方案可检测到的最小气泡的物理直径。
2. **假设陈述**：在设计解决方案前，请清楚陈述您所做的假设，例如：
   - 假设图像中的气泡大部分是球形的（在投影下为圆形）；
   - 假设气泡之间虽然可能粘连，但不会大面积重叠导致无法分割；
   - 假设光照梯度变化缓慢，可以通过形态学校正；
   - 假设忽略由于景深导致的模糊气泡的漏检风险。
3. **算法设计与实现**：基于 OpenCV（C++/Python）或 MATLAB 设计程序，实现以下功能：
   (a). 对气泡图像进行预处理，去除背景不均匀性；
   (b). 实现气泡的准确分割，尽可能将粘连气泡分开；
   (c). 计算图像中气泡总面积与图像总面积的比率；
   (d). 统计气泡的总数量，并绘制结果图（如在原图上标记出每个气泡的轮廓和编号）。

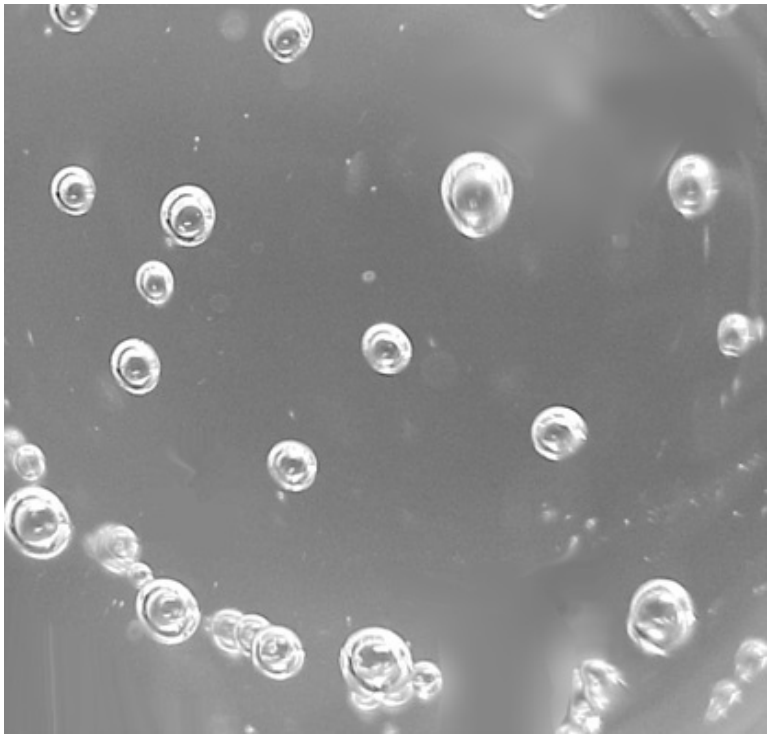


图 7.1: 图 7.1 液体中的气泡图像

# 第 8 章 附录 1：基于 OpenCV 的轮廓和轮廓特征

轮廓是一系列相连的点组成的曲线，代表了物体的基本外形，相对于边缘，轮廓是连续的，边缘并不全部连续。寻找轮廓的操作一般用于二值化图，所以通常会使用阈值分割或 Canny 边缘检测先得到二值图。OpenCV 中寻找轮廓是针对白色物体的，一定要保证物体是白色，而背景是黑色，不然很多人在寻找轮廓时会找到图片最外面的一个框。

## 8.1 寻找轮廓

调用 cv2.findContours() 函数:

根据题目要求，由问题驱动，设计图像处理算法。

```
import cv2
img = cv2.imread('handwriting.jpg')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY_INV +
    cv2.THRESH_OTSU)
# 寻找二值化图中的轮廓
image, contours, hierarchy = cv2.findContours(
thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
print(len(contours)) # 结果应该为2
```

- 参数 1：二值化原图
- 参数 2：轮廓的查找方式，一般使用 `cv2.RETR_TREE`，表示提取所有的轮廓并建立轮廓间的层级。
- 参数 3：轮廓的近似方法。
  比如对于一条直线,可以存储该直线的所有像素点（`cv2.CHAIN_APPOX_NONE`），也可以只存储起点和终点。使用 `cv2.CHAIN_APPROX_SIMPLE` 就表示用尽可能少的像素点表示轮廓.

## 8.2 绘制轮廓

调用 cv2.drawContours() 函数：

```
cv2.drawContours(img, contours, -1, (0,0,255),2)
```

其中参数 2 就是得到的 contours，参数 3 表示要绘制哪一条轮廓，-1 表示绘制所有轮廓，参数 4 是颜色（B/G/R 通道，所以 (0,0,255) 表示红色），参数 5 是线宽。

一般情况下，我们会首先获得要操作的轮廓，再进行轮廓绘制及分析：

```
cnt = contours[1]
cv2.drawContours(img, [cnt], 0, (0, 0, 255), 2)
```

## 8.3 轮廓层级

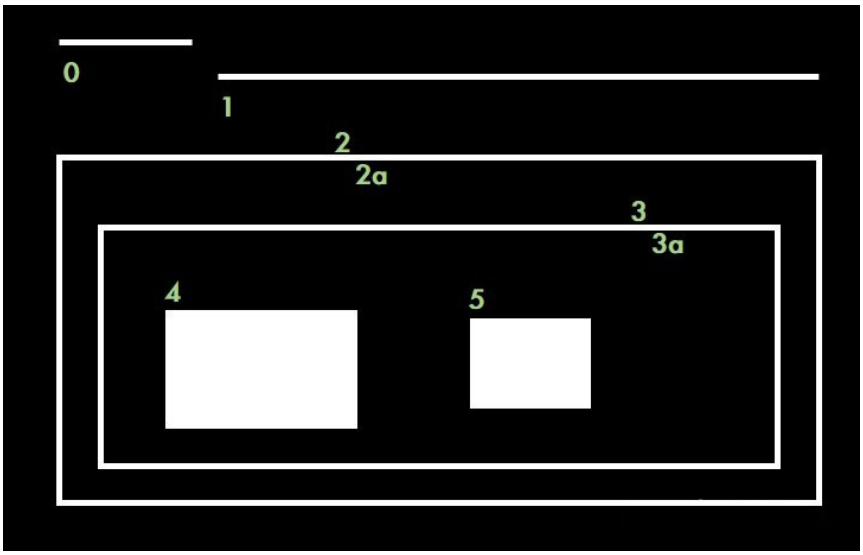



图 8.1: 轮廓层级示意图

图中总共有 8 条轮廓，2 和 2a 分别表示外层和里层的轮廓，3 和 3a 也是一样。从图中看得出来：

- 轮廓 0/1/2 是最外层的轮廓，我们可以说它们处于同一轮廓等级：0 级
- 轮廓 2a 是轮廓 2 的子轮廓，反过来说 2 是 2a 的父轮廓，轮廓 2a 算一个等级：1 级
- 同样 3 是 2a 的子轮廓，轮廓 3 处于一个等级：2 级

- 类似的，3a 是 3 的子轮廓，等等……

## OpenCV 中轮廓等级的表示

如果我们打印出 cv2.findContours() 函数的返回值 hierarchy，会发现它是一个包含 4 个值的数组：**[Next, Previous, First Child, Parent]** - Next:

- 与当前轮廓处于同一层级的下一条轮廓，没有为-1。- Previous:
- 与当前轮廓处于同一层级的上一条轮廓，没有为-1。- Firtst Child:
- 当前轮廓的第一条子轮廓，没有为-1。- Parent:
- 当前轮廓的父轮廓，没有为-1。

**表 8.1:** 轮廓层次列表（hierarchy）中四元素数组里每个元素的含义

| 索引 | 含义 |
|---|---|
| 0 | 同级的下一条轮廓 |
| 1 | 同级的上一条轮廓 |
| 2 | 下级的第一个子节点 |
| 3 | 上级的父节点 |

## 轮廓的四种寻找方式

- RETR_LIST：所有轮廓属于同一层级
- RETR_TREE: 完整建立轮廓的各属性
- RETR_EXTERNAL: 只寻找最高层级的轮廓
- RETR_CCOMP: 所有轮廓分 2 个层级，不是外界就是最里层

**例题 8.1** 问：如何把下图的三个内圈填满灰色？

代码如下：

```
import cv2
import numpy as np
img = cv2.imread('circle_ring.jpg')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_,th = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_
    OTSU)

# 寻找轮廓，使用cv2.RETR_CCOMP寻找内外轮廓
```

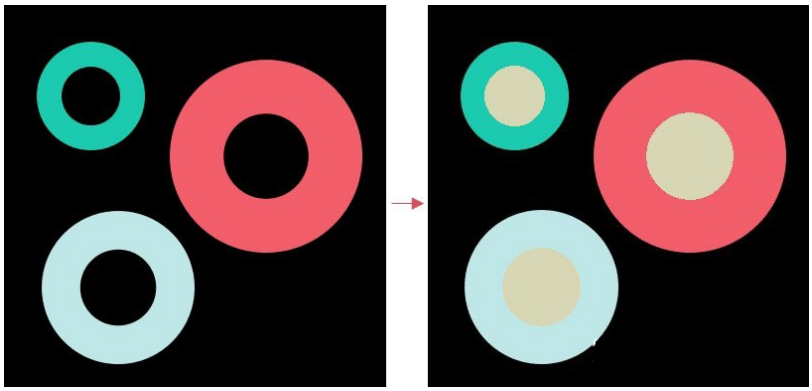


图 8.2: 附录图 1-2

```
image, contours, hierarch = cv2.findContours(th, cv2.RETR_CCOMP, 2)
# 找到内层轮廓并填充
# hierarchy的形状为(1,6,4), 使用np.squeeze压缩一维数据, 变成(6,4)
hierarchy = np.squeeze(hierarchy)

for i in range(len(contours)):
# 存在父轮廓, 说明是里层
if (hierarchy[i][3] != -1):
cv2.drawContours(img, contours, i, (180, 215, 215), -1)
cv2.imwrite('result.jpg', img)
```

## 8.4 轮廓的特征

- 计算物体的周长、面积、质心、最小外接矩形等
- OpenCV 函数:cv2.contourArea(),cv2.arcLength(),cv2.approxPolyDP() 等

### 图像矩

- 图像矩可以帮助我们计算图像的质心，面积等；
- 函数 cv2.moments() 会将计算得到的矩以字典形式返回。

```
import cv2
import numpy as np
```

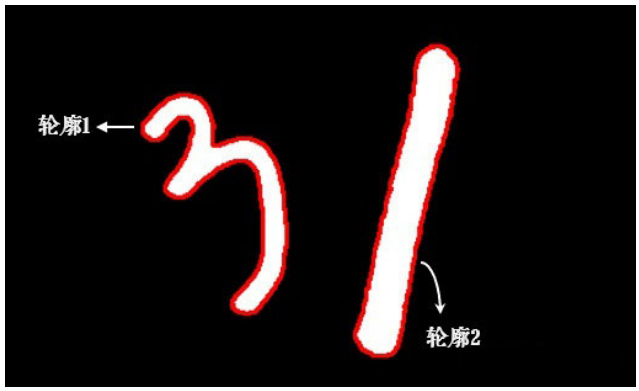



图 8.3: 附录图 1-3

```
img = cv2.imread('handwriting.jpg', 0)
_, thresh = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV + cv2.
    THRESH_OTSU)
image, contours, hierarchy = cv2.findContours(thresh, 3, 2)
# 以数字3的轮廓为例
cnt = contours[0]

img_color1 = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
img_color2 = np.copy(img_color1)
cv2.drawContours(img_color1, [cnt], 0, (0, 0, 255), 2)
cv2.imshow('img',img_color1)
cv2.waitKey(0)
```

采用图像矩

```
M = cv2.moments(cnt)
# 对象的质心
cx = int(M['m10'] / M['m00'])
cy = int(M['m01'] / M['m00'])
```

M 中包含了很多轮廓的特征信息，比如 M[‘m00’] 表示轮廓面积，与 `cv2.contourArea(` 计算结果是一样的。

## 轮廓面积

```
area = cv2.contourArea(cnt)
```

注意轮廓特征计算的结果并不等同于像素点的个数，而是根据几何方法算出来的，所以有小数。

笔记 如果统计二值图中像素点个数,应尽量避免循环,可以使用 cv2.countNonZero(),更加高效。

## 轮廓周长

```
perimeter = cv2.arcLength(cnt, True)
```

参数 2 表示轮廓是否封闭，显然我们的轮廓是封闭的，所以是 True。

## 外接矩形

- 形状的外接矩形有两种，如下图，绿色的叫外接矩形，表示不考虑旋转并且能包含整个轮廓的矩形。蓝色的叫最小外接矩，考虑了旋转：

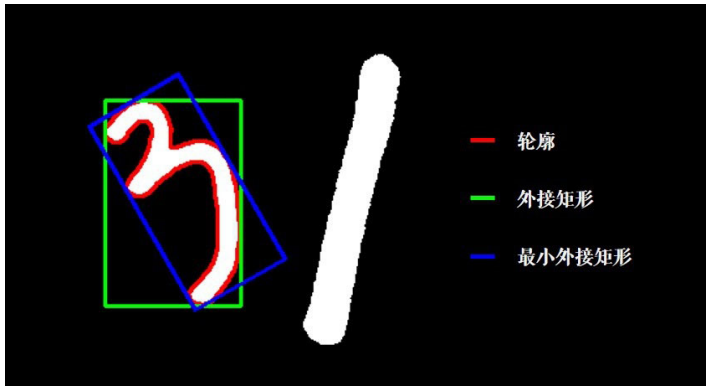



图 8.4: 附录图 1-4

```
# 外接矩形:
x, y, w, h = cv2.boundingRect(cnt)
# 最小外接矩形:
rect = cv2.minAreaRect(cnt)
# 矩形四个角点取整
box = np.int0(cv2.boxPoints(rect))
cv2.drawContours(img_color1, [box], 0, (255, 0, 0), 2)
```

`np.int0(x)` 是把 x 取整的操作,比如 377.93 就会变成 377,也可以用 `x.astype(np.int`

## 最小外接圆

```
(x, y), radius = cv2.minEnclosingCircle(cnt)
(x, y, radius) = cv2.int0((x, y, radius))
cv2.circle(img_color2, (x,y), radius, (0, 0, 255), 2)
```

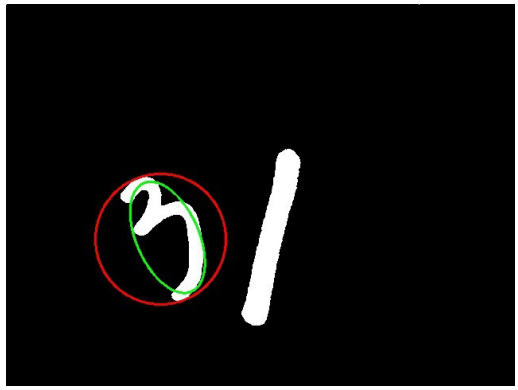


图 8.5: 附录图 1-5

## 拟合椭圆

```
ellipse = cv2.fitEllipse(cnt)
cv2.ellipse(img_color2, ellipse, (255,255,0), 2)
```

## 形状匹配

- cv2.matchShapes() 可以检测两个形状之间的相似度，返回值越小，越相似

```
img = cv2.imread('shapes.jpg', 0)
_, thresh = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_
    OTSU)
image, contours, hierarchy = cv2.findContours(thresh, 3, 2)
img_color = cv2.cvtColor(thresh, cv2.COLOR_GRAY2BGR) # 用于绘制的彩色图

# 图中有3条轮廓，我们用A/B/C表示:
cnt_a, cnt_b, cnt_c = contours[0], contours[1], contours[2]
print(cv2.matchShapes(cnt_b, cnt_b, 1, 0.0)) # 0.0
```

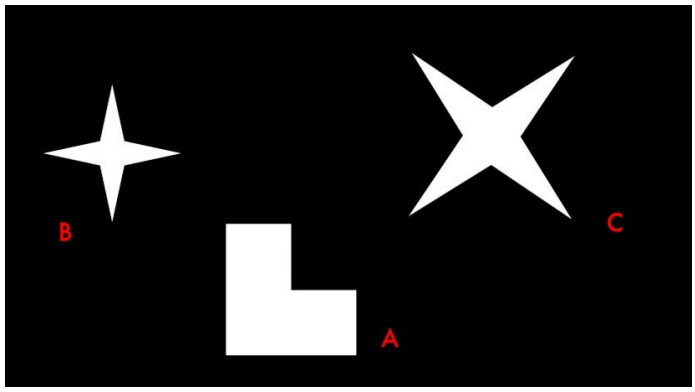



图 8.6: 附录图 1-6

```
print(cv2.matchShapes(cnt_b, cnt_c, 1, 0.0)) # 2.17e-05
print(cv2.matchShapes(cnt_b, cnt_a, 1, 0.0)) # 0.418
```

可以看到 BC 相似程度比 AB 高很多，并且图形的旋转或缩放并没有影响。其中，参数 3 是匹配方法，参数 4 是 OpenCV 的预留参数，暂时没有实现，可以不用理会。

## 轮廓近似

- 将轮廓形状近似到另外一种由更少点组成的轮廓形状，新轮廓的点的数目由我们设定的准确度来决定，用的 Douglas-Peucker 算法。

```
import cv2
import numpy as np

# 1.先找到轮廓
img = cv2.imread('unregular.jpg', 0)
_, thresh = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_
    OTSU)
image, conturs, hierarchy = cv2.findContours(thresh, 3, 2)
cnt = contours[0]

# 2.进行多边形逼近，得到多边形的角点
approx = cv2.approxPolyDP(cnt, 3, True)

# 3.画出多边形
```

```
image = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
cv2.polylines(image, [approx], True, (0, 255, 0), 2)
```

其中 cv2.approxPolyDP() 的参数 2($epsilon$) 是一个距离值，表示多边形的轮廓接近实际轮廓的程度，值越小，越精确；参数 3 表示是否闭合。

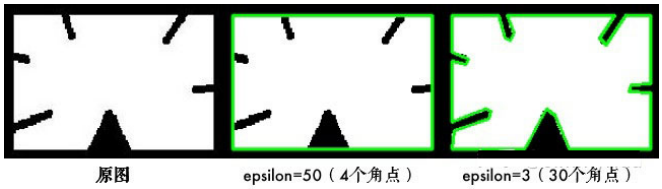



图 8.7: 附录图 1-7

## 凸包

- 凸包跟多边形逼近很像，只不过它是物体最外层的” 凸” 多边形：集合 A 内连接任意两个点的直线都在 A 的内部，则称集合 A 是凸形的。如下图，红色的部分为手掌的凸包，双箭头部分表示凸缺陷 (Convexity Defects)，凸缺陷常用来进行手势识别等。

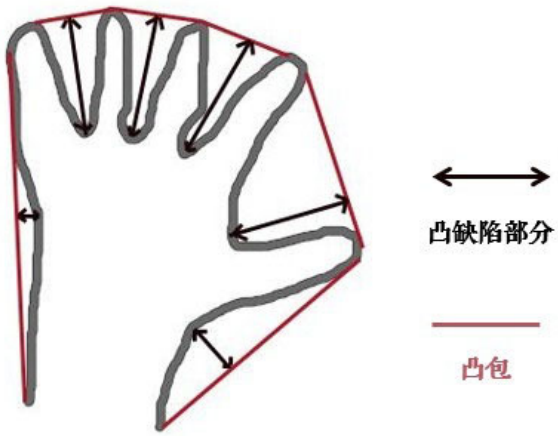



图 8.8: 凸包

```
import cv2
img = cv2.imread('convex.jpg', 0)
_, th = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY+cv2.THRESH_OTSU)
image, contours, hierarchy = cv2.findContours(th, 3, 2)
cnt = contours[0]
```

```
image = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.drawContours(image, contours, -1, (0, 0 , 255), 2)

# 寻找凸包，得到凸包的角点
hull = cv2.convexHull(cnt)

# 绘制凸包
cv2.polylines(image, [hull], True, (0, 255, 0), 2)
```

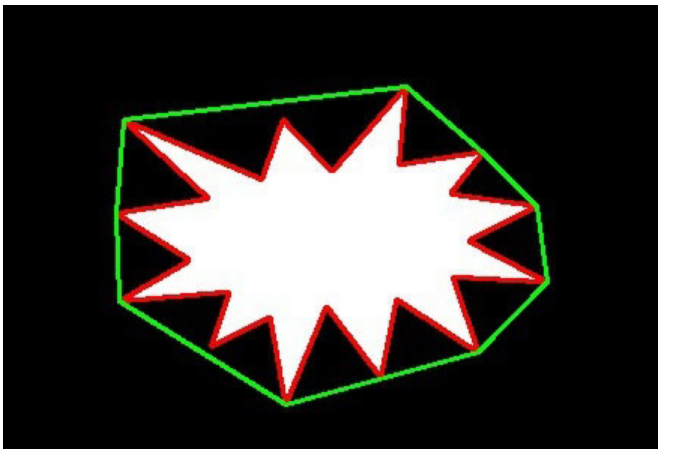


图 8.9: 凸包

其中函数 cv2.convexHull() 有个可选参数 returnPoints，默认是 True，代表返回角点的 x/y 坐标；如果为 False 的话，表示返回轮廓中是凸包角点的索引，比如说：

```
print(hull[0]) # [[362 184]]（坐标）
hull2 = cv2.convexHull(cnt, returnPoints=False)
print(hull2[0]) # [510]（cnt中的索引）
print(cnt[510]) # [[362 184]]
```

当使用 cv2.convexityDefects() 计算凸包缺陷时，returnPoints 需为 False

另外可以用下面的语句来判断轮廓是否是凸形的：

```
print(cv2.isContourConvex(hull)) # True
```

## 凸面缺陷

- OpenCV 提供了现成的函数来做这个，`cv2.convexityDefects()`.

- 注意：我们要传 returnPointsFalse 来找凸形外壳。
- 它返回了一个数组，每行包含这些值：**[start point, end point, farthest point, approximate distance to farthest point]**. 我们可以用图像来显示他们。我们画根线把 start point 和 end point 连起来。然后画一个圆在最远点。记住最前三个返回值是 cnt 的索引，所以我们我们得从 cnt 里拿出这些值.

```
import cv2
import numpy as np

img = cv2.imread('star.jpg')
img_gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(img_gray, 127, 255,0)
_,contours,hierarchy = cv2.findContours(thresh,2,1)
cnt = contours[2]

# 返回凸包角点的索引
hull = cv2.convexHull(cnt,returnPoints = False)
# 检测凸凹陷
defects = cv2.convexityDefects(cnt,hull)
# 可视化
for i in range(defects.shape[0]):
s,e,f,d = defects[i,0]
start = tuple(cnt[s][0])
end = tuple(cnt[e][0])
far = tuple(cnt[f][0])
cv2.line(img,start,end,[0,255,0],2)
cv2.circle(img,far,5,[0,0,255],-1)
```

## 点到轮廓距离

- cv2.pointPolygonTest() 函数计算点到轮廓的最短距离（也就是垂线），又称多边形测试：

```
dist = cv2.pointPolygonTest(cnt, (100, 100), True) # -3.53
```

其中参数 3 为 True 时表示计算距离值：点在轮廓外面值为负，点在轮廓上值为 0，点在轮廓里面值为正；参数 3 为 False 时，只返回-1/0/1 表示点相对轮

廓的位置，不计算距离。